

Report on Internship Project

TINYML FOR EDGE PROCESSING

Submitted By

Ahan Halder (2310329)

B.Tech / EEE / II

National Institute of Technology Karnataka, Surathkal

as part of the requirements of

Summer Internship [2025]

under the guidance of

Dr. Sowmya Kamath S., Dept of IT, NITK Surathkal

undergone at



HEALTHCARE ANALYTICS AND LANGUAGE ENGINEERING (HALE) LAB

DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA, SURATHKAL

May-Jul 2025

HEALTHCARE ANALYTICS AND LANGUAGE ENGINEERING (HALE) LAB
Dept. of Information Technology
National Institute of Technology Karnataka, Surathkal

C E R T I F I C A T E

This is to certify that the Internship Report entitled “**TinyML for Edge Processing**” is submitted by the student mentioned below -

Details of the Intern

Intern Name	Register No.	Branch/Institute	Signature
Ahan Halder	2310329	EEE / National Institute of Technology Karnataka, Surathkal	

This report is a record of the work carried out by the Intern as part of the **Summer Internship** during the summer term of the year **2025**, as part of a research project offered by the Healthcare Analytics and Language Engineering (HALE) Lab. The duration of the internship was **19-05-2025** to **18-07-2025**.

Name and Signature of Internship Guide (with date)
Dr. Sowmya Kamath S.

DECLARATION

I hereby declare that the project report entitled “**TinyML for Edge Processing**” submitted by me as part of the **Summer Internship project** during the **Summer Term of 2025**, is my original work. I declare that the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles elsewhere.

Details of the Intern

Intern Name	Register No.	Branch/Institute	Signature
Ahan Halder	2310329	EEE / National Institute of Technology Karnataka, Surathkal	

Place: NITK Surathkal

Date: 18-07-2025

TinyML for Edge Processing

Ahan Halder, National Institute of Technology Karnataka, Surathkal

I. INTRODUCTION

Tiny Machine Learning (TinyML) has been a revolutionary solution to deploying innovative systems in environments where resources are limited. By supporting machine learning inference on microcontroller units (MCUs) with constrained memory and computational capabilities, TinyML brings analytics to the edge, removing reliance on cloud infrastructure and enabling realtime decision-making with low latency and power usage. This is especially useful in applications like environmental monitoring, where access to central processing might be impossible or unreliable.

This project involved designing, optimizing, and simulating TinyML models for water quality monitoring with edge-based sensors. The objective was to develop a light, scalable pipeline with the capability of processing sensor data locally and making predictions on water parameters like pH, turbidity, and conductivity. Such insights are vital in determining unsafe conditions for water in remote or distributed environments.

To do this, several machine learning models were implemented, such as classification and regression models, and anomaly detection with LSTM autoencoders and bidirectional models. Representative sensor data was used to train the models and optimize them using quantization and pruning methods to minimize computational requirements and memory footprint—facilitating deployment on virtual ESP32 MCUs. The methods allowed to do predictive performance while making models ready for realtime inference on low-power hardware.

The deployment process combined both edge and cloud elements. Model predictions and sensor data were maintained in a Firebase Realtime Database for realtime observation. Simultaneously, raw input data and logs were stored in Google Cloud Buckets for testing and verification support. A minimalistic dashboard, built with HTML, CSS, and JavaScript, delivered dynamic visualization of real-time sensor readings and model predictions to facilitate easy real-time observation of water quality status.

With the use of TinyML for field-deployable environmental sensing, this system provides an actionable and scalable solution for smart water monitoring. It minimizes data transmission needs, allows for timely detection of anomalies, and facilitates optimal use of compute resources—helping to catalyze a larger movement toward sustainable, autonomous sensing systems under the paradigm of edge computing.

II. RELATED WORKS

Design of Li River Water Quality Dynamic Monitoring System Based on Raspberry Pi and TinyML

- system is developed for the realization of real-time watershed detection.
- software part implements image recognition with the yolo-tiny model, and utilizes cloud database and cloud object storage return to visualize the data, pictures, and identification tags collected on the Raspberry Pi side
- To sort out the types of trash on the water, TinyML technology was used to identify the types of trash on the water.
- Yolo-Tiny Model Commonly used image recognition models are CNN convolutional models. CNN model is divided into convolutional layers, a pooling layer and a fully connected layer. The image can be represented as a matrix consisting of 0 and 1. The convolution kernel and the corresponding matrix are multiplied and summed to obtain a new matrix; the pooling layer compresses the neural network, and the fully connected layer classifies the features after they are extracted, and finally recognizes the image.
- cloud database performs operations such as building and deploying through the cloud platform and the project interacts with a relational database MYSQL.

An energy efficient TinyML model for a water potability classification problem

- (1) it considers only data that can be acquired by electronic sensors, (2) it does not need to use the Cloud for its execution.
- compared with a similar model functioning in the Cloud, the proposed model requires 51.2% less memory space, performs all inference tests approximately 99.95% faster, and consumes about 99.95% less energy.
- pH value (can be obtained using the PH4502C1 sensor, for example); Hardness (can be obtained using the WHB-3002 sensor, for example); Solids or Total Dissolved Solids — TDS (can be obtained using the CDPB-033 sensor, for example); Chloramines (can be obtained using the PC1014 sensor, for example); Conductivity (can be obtained using the CDPB-035 sensor, for example); Turbidity (can be obtained using the CUS52D6 sensor, for example).
- During the data preprocessing phase, unwanted columns (non electronic data) were removed, and missing values were addressed by filling nulls with the average value of their respective columns
- a random sampling of non-potable data was conducted to equalize the sample sizes and achieve balance.
- Sci-kit Learn. To identify the best hyperparameters for the model, the Optuna tool was used during training.

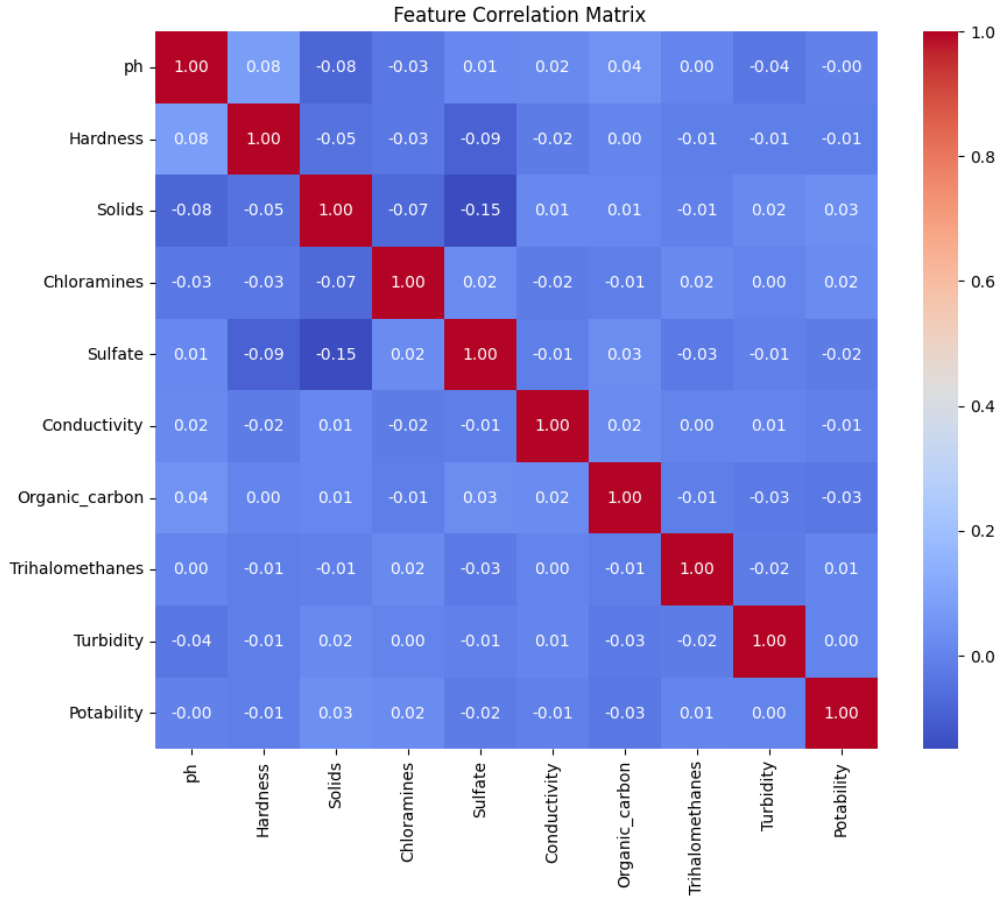


Fig. 1: Correlation Matrix of Features in water_potability.csv

III. METHODOLOGY

A. Datasets & Pre-processing

TABLE I: Parameters of water_potability.csv

Parameter	Description
pH	Measures the acidity or alkalinity of water. The ideal range for drinking water is 6.5 to 8.5.
Hardness	Concentration of calcium and magnesium ions; affects taste and scaling.
Solids	Total dissolved solids (ppm); impacts taste, texture, and usability.
Chloramines	Disinfectant level for microbial control.
Sulfate	Influences taste; excessive levels may cause health issues.
Conductivity	Reflects ion concentration; indicates the water's ability to conduct electricity.
Organic Carbon	Represents natural organic material; associated with microbial activity.
Trihalomethanes	By-products of chlorination; elevated levels may pose health risks.
Turbidity	Indicates water clarity; high values may suggest contamination.
Potability	Binary label: 0 = Not potable, 1 = Potable.

1) water_potability.csv:

water_potability.csv This dataset used is made up of 3,276 entries and 10 parameters, as detailed in Table I. Each record represents a water sample described by the 9 parameters and a binary potability label indicating whether the sample is potable or not.

Missing Values: The dataset contained missing values in the 3 following attributes:

- **pH:** 491 missing values (15%)
- **Sulfate:** 781 missing values (23.8%)
- **Trihalomethanes:** 162 missing values (4.9%)

Missing values were handled by *mean imputation*, ensuring consistency without majorly changing the data distribution as a whole.

Class Imbalance: The initial class distribution exhibited moderate imbalance:

- **Not Potable (0):** 1,998 samples (61%)
- **Potable (1):** 1,278 samples (39%)

To counter this imbalance and enhance model generalization, the *Synthetic Minority Over-sampling Technique (SMOTE)* was used in the training set. Post-SMOTE distribution:

- **Class 0:** 1,586 samples
- **Class 1:** 1,586 samples

This ensured balanced class representation and reduced bias.

Feature Scaling: Features were standardized using *z-score normalization*. Standardization ensures that all features contribute equally while training and inferring the model.

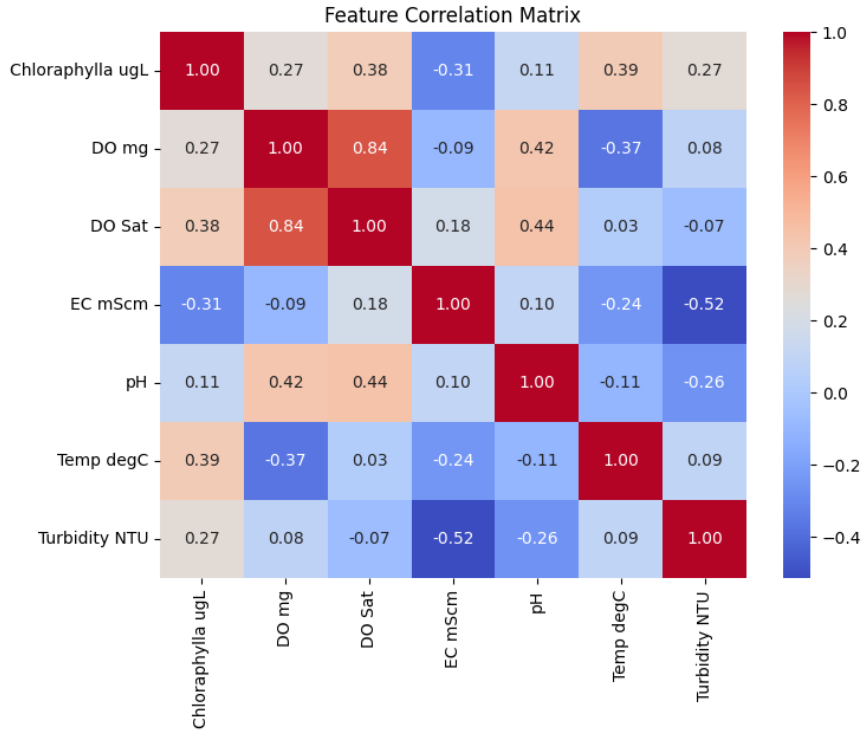


Fig. 2: Correlation Matrix of Features in Hourly_data.csv

TABLE II: Parameters of Hourly_data.csv

Parameter	Description
Chlorophylla ugL	Concentration of chlorophyll-a in micrograms per liter; an indicator of algal biomass.
DO mg	Dissolved Oxygen in milligrams per liter; vital for aquatic organisms' respiration.
DO Sat	Dissolved Oxygen Saturation percentage; shows how oxygenated the water is.
EC mScm	Electrical Conductivity in milliSiemens per centimeter; reflects ionic concentration.
pH	Measures water acidity or alkalinity; important for chemical and biological stability.
Temp degC	Water temperature in degrees Celsius; affects solubility and metabolic rates.
Turbidity NTU	Turbidity in Nephelometric Turbidity Units; indicates water clarity and potential contamination.

2) *Hourly_data.csv*: [Hourly_data.csv](#) This dataset used is made up of 39,958 entries and 7 variables, as detailed in Table II. Each record represents water quality parameters based on sensor data collected at an hourly frequency.

No Missing Values: All columns are fully occupied, which makes it easier to proceed further with preprocessing and modeling.

Based on the correlation matrix, the feature with the strongest average correlation to others is DO Sat (Dissolved Oxygen Saturation %). This suggests that DO Sat might be a strong indicator of water quality.

The absence of specific labels implies that this dataset is more suited towards unsupervised exploration or target-driven regression using DO Sat.

Applied Z-score standardization:

- Centers the data (mean = 0).

- Scales to unit variance (std = 1).

fit_transform on training data scales the parameters based on mean and standard deviation.

transform on testing data ensures that the transformation is based on the same methods.

Purpose: makes sure that the features are on a comparable scale, which is important for algorithms like linear regression, SVM, or gradient-based models.

B. Models

1) *Binary Classification Model*: In order to forecast water potability, a light feedforward neural network was realized with the Sequential API. It has three hidden layers with 16, 8, and 4 neurons respectively. Every layer uses the Rectified Linear Unit (ReLU) activation function and applies L2 regularization in an effort to reduce overfitting and improve generalization. The last layer employs a sigmoid activation function to output binary results, with the goal of distinguishing between potable and non-potable water samples. The model was trained utilizing the Adam optimizer and binary cross-entropy as the loss function.

For improved model strength and avoiding overfitting, early stopping was used with five epochs of patience. This made training stop as soon as validation loss flattened out, keeping the best weights seen along the way. The model was trained on an SMOTE-synthetic balanced dataset and tested on a 20% test split. Real-time validation metrics feedback monitored training convergence and permitted dynamic adaptations.

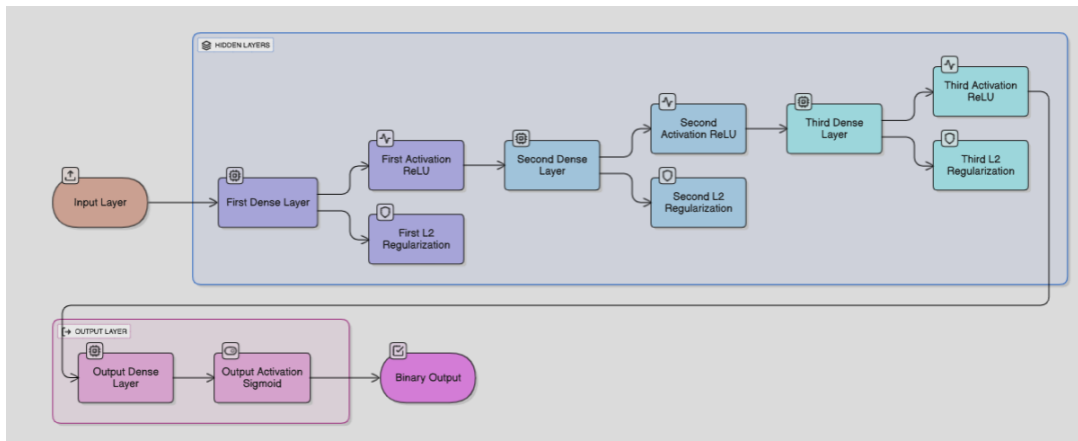


Fig. 3: Block Diagram for Model 1

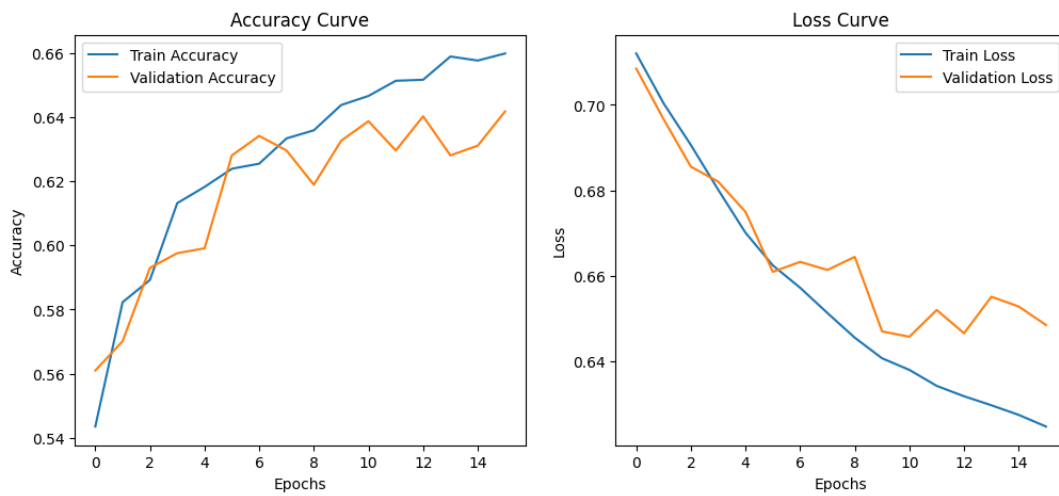


Fig. 4: Accuracy Curve and Loss Curve for Model 1

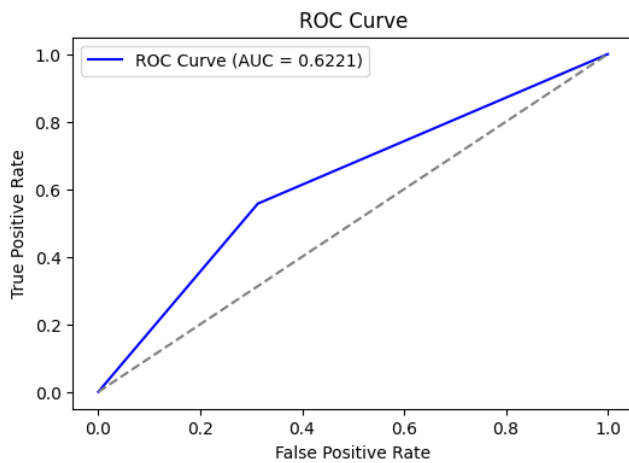


Fig. 5: ROC Curve for Model 1

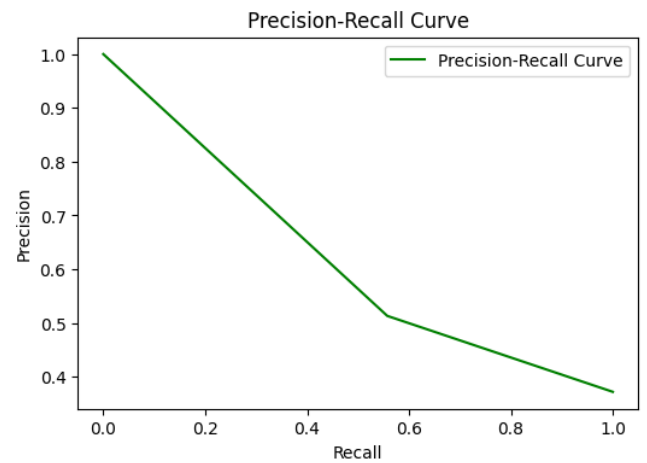


Fig. 6: Precision-Recall Curve for Model 1

Evaluation Results At training completion, the model showed good performance on unobserved test data. Accuracy, precision, recall, F1-score, and ROC-AUC scores were calculated to provide a wider perspective of classification

reliability. Furthermore, a confusion matrix plot indicated the model's capability to accurately distinguish between potable and non-potable samples. Accuracy and loss curves plotted against consecutive epochs indicated that learning did

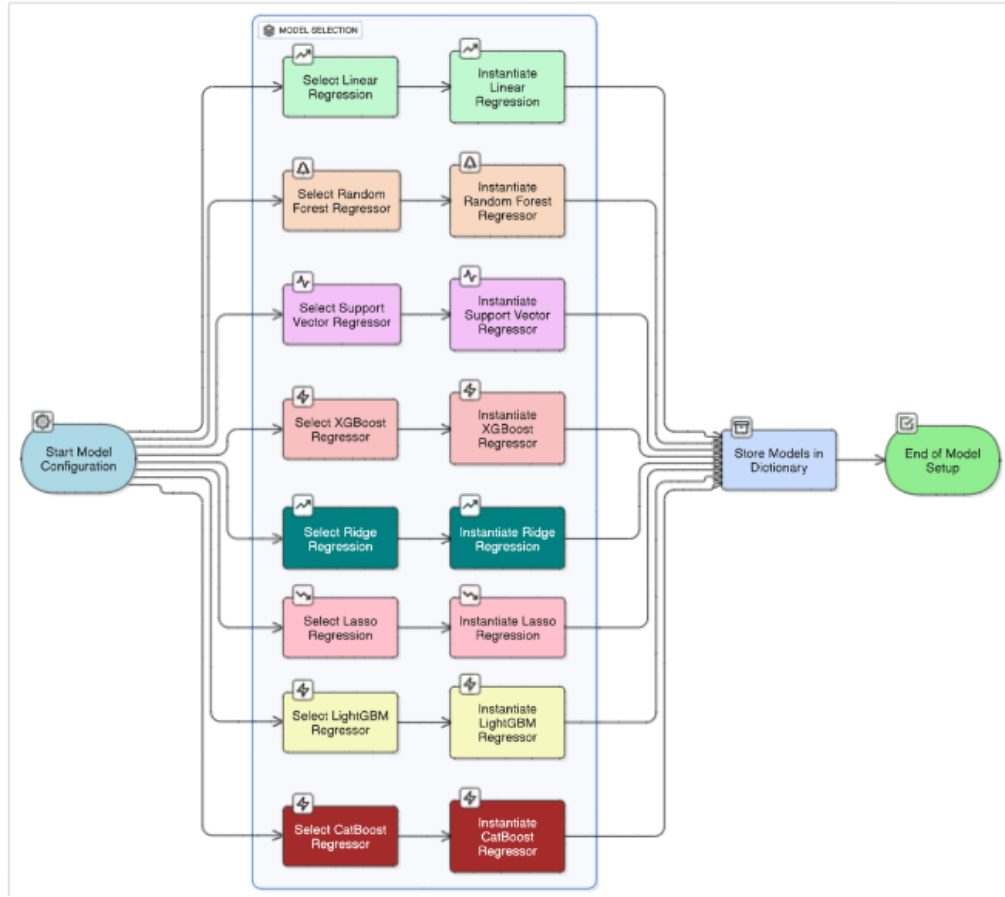


Fig. 7: Block Diagram for Model 2

proceed smoothly and stabilize without deterioration.

To facilitate real-time deployment in low-resource settings, the trained model of the neural network was successfully quantized to TensorFlow Lite with default post-training quantization. This was achieved to get a small .tflite file of approximately 3.74 KB size, which is a sufficiently small size for utilization on Microcontroller Units (MCUs) and other embedded platforms with limited memory and computation capacities. The light model structure, together with low storage needs and low inference overhead, provides efficient integration into edge-based water monitoring systems.

2) Regression Models: In this research, several regression models were investigated for the prediction of a chosen target variable (DO Sat) from hour-based sensor or environmental data. The aim was to benchmark various algorithmic approaches and determine an optimal model that meets the criteria of prediction accuracy, generalization, and deployment efficacy.

Linear Regression (Baseline): A traditional method presuming a linear correlation between the features and target. This model was used as a baseline for comparison. Although interpretable and simple, its performance was restricted by not being able to learn non-linear patterns present in the dataset.

Support Vector Regression (SVR): Used with a radial basis function (RBF) kernel, SVR performed well on

non-linear dependencies. Hyperparameter tuning of $C=1.0$ and $\epsilon=0.1$ provided controlled regularization and error bounds. SVR provided the worst performance of all the regressors considered and based on our evaluation metrics.

Random Forest Regressor: A strong model with 100 decision trees to extract interactions between features and non-linear patterns. It was resistant to outliers and showed high accuracy with little tuning. The importance of features in this model provided useful information on the predictions. This was the best regressor as can be seen in Fig 8.

XGBoost Regressor: An advanced gradient-boosting framework known for speed and regularization capabilities. With $n_estimators=100$ and $learning_rate=0.1$, it achieved low error rates and strong generalization. XGBoost also facilitated feature importance analysis and provided rapid inference for deployment.

Ridge & Lasso Regression: These regularized linear regression models were thought to reduce overfitting and provide better interpretability but much to despair.

- Ridge Regression is used in a situation where independent variables are highly correlated, by adding a penalty term to the cost function.
- Lasso Regression applies L1 regularization and performs variable selection.

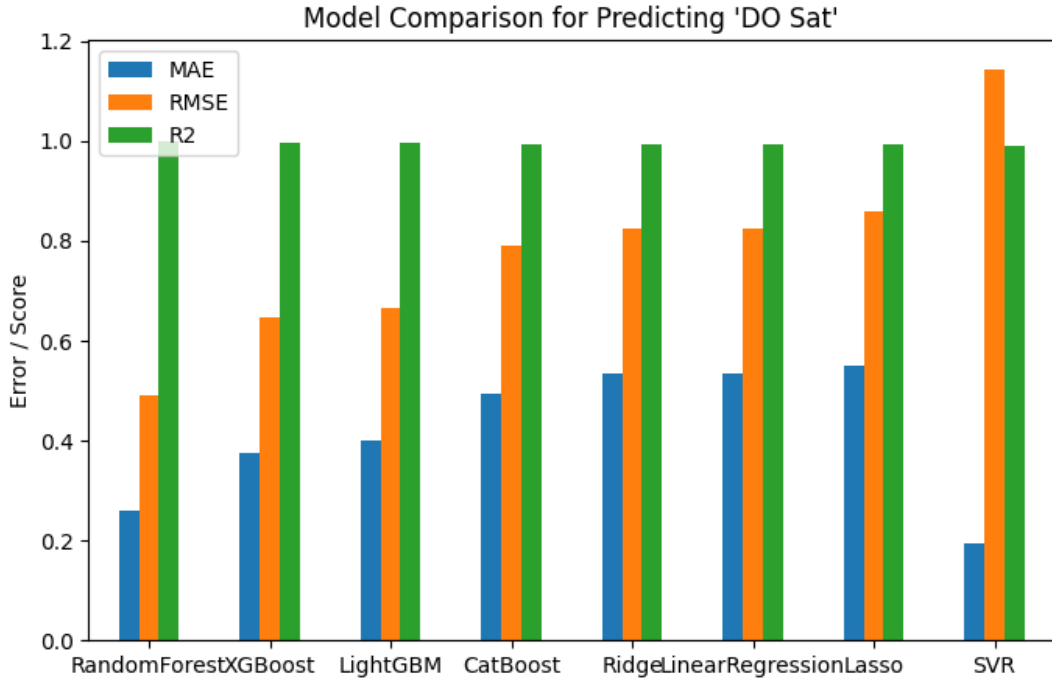


Fig. 8: Comparison Plots for different Regressors in Model 2

Keras-Based Linear Model: A lightweight neural regressor was trained using TensorFlow and converted to a quantized TFLite format. This allowed for:

- Drastic reduction in model size (KB scale)
- Efficient edge inference with latency under 1 ms/sample

Model Comparison: Models were evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Coefficient of Determination (R^2)

Among all models tested, the Random Forest Regressor consistently delivered the most accurate results. Trained on scaled data with 100 estimators, it picked up intricate relations between features while being robust to outliers and noise. The model performed exceptionally well with a Mean Absolute Error (MAE) of 0.259, a Root Mean Squared Error (RMSE) of 0.489, and an R^2 score of 0.998, reflecting near-perfect fit and generalization ability. These findings identified Random Forest as the most robust model for the data provided.

To accommodate edge computing's limitations, a lightweight regression model was implemented with Keras and then translated into TensorFlow Lite with default quantization methods. Even with its small architecture, the quantized model's predictive accuracy remained steady while significantly lowering its memory size. The resulting TFLite model was roughly 1.5 KB in size, and inference latency was between 0.005 to 0.008 milliseconds per sample, allowing for ultra-quick predictions on low-resource hardware. This renders the model suitable for real-time use in TinyML applications.

3) *Autoencoder LSTM Model:* To preserve time dynamics, the data is restructured into 24-hour sequences of windows—an approach that enables the model to learn hourly fluctuations and trends. These sequences are the input to the LSTM autoencoder, which compresses and reconstructs the sequences. The encoder part of the model is comprised of stacked LSTM layers that learn latent representations of the input sequences. The decoder then tries to recover the original inputs based on a repeat vector and symmetric LSTM layers followed by a time-distributed dense output layer.

TABLE III: Autoencoder LSTM Model 1

Layer (type)	Output Shape	Param #
LSTM	(None, 24, 64)	18,432
LSTM (lstm_1)	(None, 32)	12,416
RepeatVector	(None, 24, 32)	0
LSTM (lstm_2)	(None, 24, 32)	8,320
LSTM (lstm_3)	(None, 24, 64)	24,832
TimeDistributed	(None, 24, 7)	455

The model is trained unsupervised with mean squared error (MSE) as the loss function, optimized with Adam. Reconstruction errors are calculated after training to test how well each sequence was reconstructed. Anomaly threshold is defined at the 95th percentile of MSE values—sequences beyond this are considered anomalous. The results are plotted in a figure of reconstruction error on all sequences, with a red line representing the anomaly threshold.

The error graph of reconstruction shows a number of useful things regarding the time dynamics and stability of

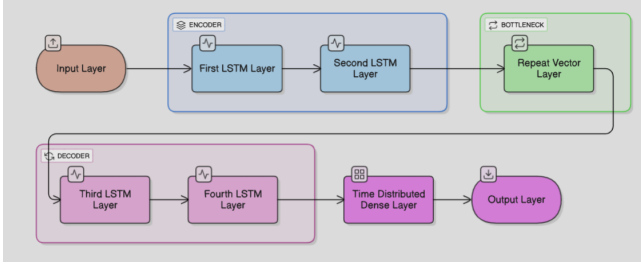


Fig. 9: Autoencoder LSTM Model 1 Block Diagram

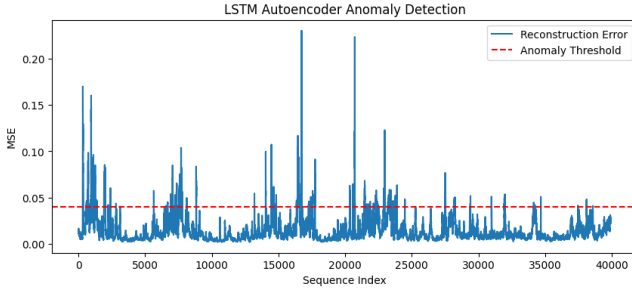


Fig. 11: Autoencoder LSTM Model 1 Reconstruction

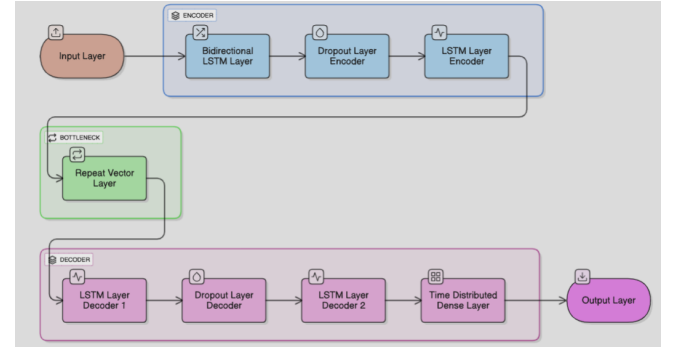


Fig. 10: Autoencoder LSTM Model 2 (Bidirectional) Block Diagram

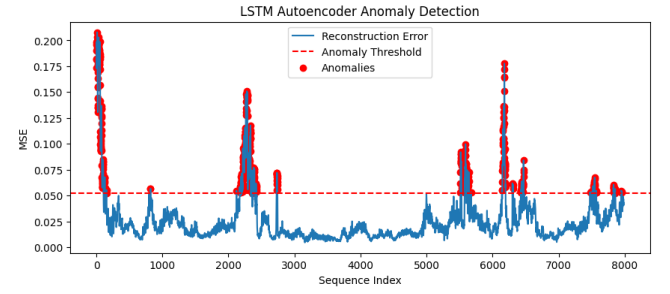


Fig. 12: Autoencoder LSTM Model 2 (Bidirectional) Reconstruction

water quality data set.

Normal vs. Abnormal Sequences

- Areas of low MSE on the graph show sequences that LSTM autoencoder was capable of reconstructing extremely well — these are normal or well-represented in the training set.
- Peaks in MSE signal epochs where the input data were well off learned patterns — these are indicated as possible anomalies.

Anomaly Threshold Effect

- The horizontal red dashed line indicates the threshold — here, at the 95th percentile of all reconstruction errors.
- Any point crossing over the line is assumed to be anomalous, i.e., the model could not replicate it with its trained latent representation. This may be equivalent to a sensor malfunction, abrupt environmental interference, or unusual water behavior.

Model 2 adds a Bidirectional LSTM (BiLSTM) to the encoder, so that it can learn from both the past and the future context in every 24-hour sequence. This is different from Model 1's unidirectional LSTM, which processes data sequentially forward in time. With access to the entire temporal domain, Model 2 achieves richer contextual insight into feature dynamics—essential for fine-grained anomaly detection in environmental data sets.

Moreover, Dropout layers are inserted in Model 2's encoder and decoder. This regularization method prevents overfitting by discarding units at random during training, forcing

TABLE IV: Autoencoder LSTM Model 2 (Bidirectional)

Layer (type)	Output Shape	Param #
Bidirectional (bidi- rectional_2)	(None, 24, 128)	36,864
Dropout (dropout_4)	(None, 24, 128)	0
LSTM (lstm_13)	(None, 32)	20,608
RepeatVector (repeat_vector_3)	(None, 24, 32)	0
LSTM (lstm_14)	(None, 24, 32)	8,320
Dropout (dropout_5)	(None, 24, 32)	0
LSTM (lstm_15)	(None, 24, 64)	24,832
TimeDistributed (time_distributed_3)	(None, 24, 7)	455

the model to acquire more generalizable representations. Model 1 has no regularization method implemented, which could weaken it towards memorization of patterns unique to the training set.

Model 1 employs relu activations, which are more computationally efficient but can be weakened by vanishing gradients in large recurrent networks. Model 2 defaults to tanh, a more stable option for LSTM models, particularly when dealing with reconstruction of continuous data with minute fluctuations—such as pH or DO levels.

Furthermore, Model 2 utilizes EarlyStopping, which stops training when validation performance no longer improves.

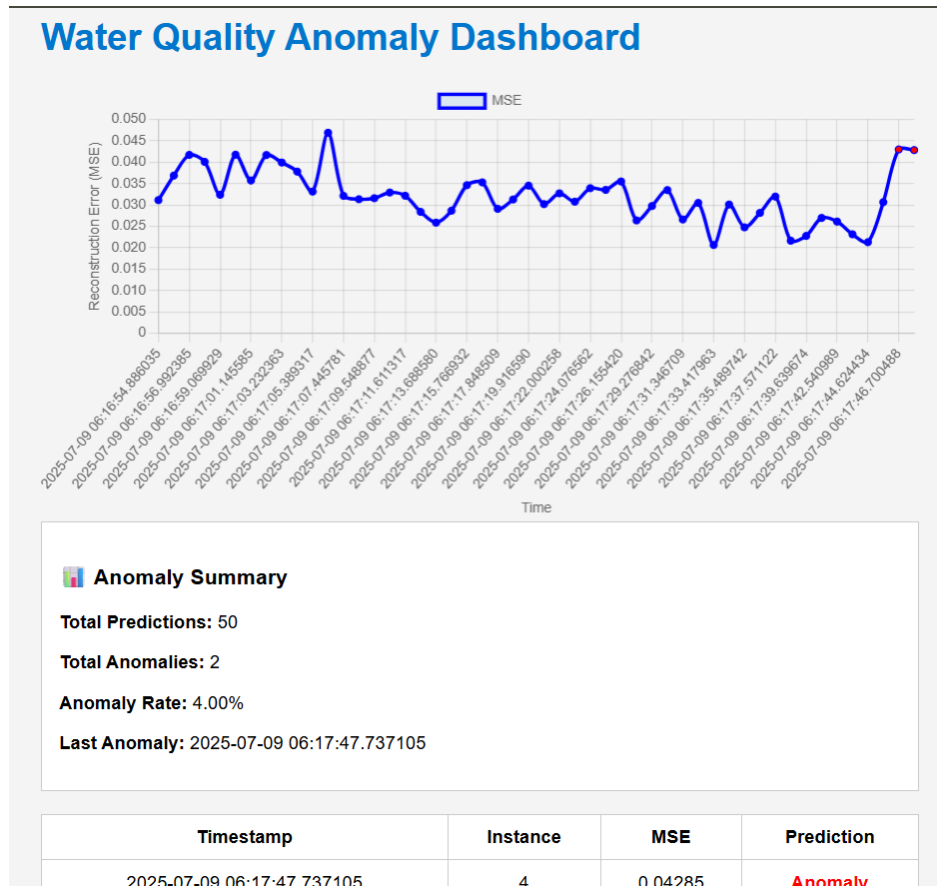


Fig. 13: Dashboard

This not only accelerates training but also guarantees the optimal generalization point selection. Model 1 trains for a predetermined 50 epochs irrespective of validation loss trends, potentially causing overfitting.

The LSTM autoencoder, by design, does not predict future values of water quality parameters—it reconstructs input sequences to learn what "normal" looks like and flags anomalies when reconstruction error exceeds a threshold. While this unsupervised setup is powerful for pattern recognition, it lacks direct forecasting capability. However, the model can be further developed by coupling it with sequence-to-one regression models, such as unidirectional LSTMs or GRUs, that explicitly predict the next timestep value for a selected parameter. Additionally, ensemble strategies—combining the autoencoder's anomaly signals with forecasts from models like XGBoost or SVR—can produce hybrid systems that both anticipate and explain deviations.

IV. EXPERIMENTAL SETUP & SIMULATION

In an ideal deployment scenario, the TensorFlow Lite model would be embedded onto a microcontroller unit (MCU) like an ARM Cortex-M or ESP32. It would be part of a water monitoring IoT system. The model would take real-time sensor inputs, which are preprocessed locally to fit the expected input format, such as scaled and sequenced data. It would then carry out anomaly detection completely on-

device. By using optimized TFLite operations and a smaller model size, the MCU could determine reconstruction error and classify sequences without needing a cloud connection. This setup would be lightweight and power-efficient, offering immediate anomaly alerts through onboard LEDs, GSM modules, or edge dashboards. It would enable reliable and timely environmental monitoring in remote or resource-limited areas.

In the simulation setup, Google Cloud Platform (GCP) was essential for enabling scalable, distributed anomaly detection with TensorFlow Lite models. The main idea was to mimic how multiple microcontroller units could work together within a sensor network. Instead of using physical devices, the simulation utilized GCP's virtualized infrastructure to run several interpreters at the same time, each representing a separate edge device. This allowed for efficient testing of collaborative inference logic without the limitations of hardware deployment.

The Firebase Realtime Database, served as the main logging and coordination hub. Each simulated device sent its prediction results to a shared Firebase path, including timestamp, instance ID, reconstruction error (MSE), and anomaly classification. This setup allowed for real-time monitoring and traceability of model behavior across all virtual instances. Using Firebase also made data synchronization and remote access easier, which was ideal for collaborative

research or dashboard integration.

The simulation loop was carried out in a GCP-hosted environment, Colab notebook providing reliable execution and access to cloud resources. The dynamic thresholding mechanism recalibrated anomaly sensitivity based on recent RMSE trends. While it was computed locally, it was logged remotely, showing how GCP can support adaptive intelligence in edge-like deployments. Overall, GCP created the framework for simulating a distributed, cloud-connected anomaly detection system that resembles real-world IoT deployments in monitoring water quality.

To enhance the cloud-based simulation, a custom dashboard was created using HTML, CSS, and JavaScript. This dashboard provided an easy-to-use, real-time visualization interface for the anomaly detection results stored in Firebase. It acted as a dynamic front end, allowing researchers and collaborators to monitor model activity without having to deal with raw JSON data or back-end queries.

The dashboard connected to the Firebase Realtime Database and showed key metrics like sequence index, MSE values, anomaly status, dynamic threshold, and timestamp for each prediction. JavaScript facilitated asynchronous data fetching and live updates via Firebase's client SDK, ensuring that new entries showed up immediately as they were added from the simulation. CSS styling created a clean, responsive layout, using visual cues like color-coded labels to differentiate between "Normal" and "Anomaly" classifications.

This dashboard not only connected the simulation logic with user-friendly monitoring but also showed how edge AI deployments can be improved with cloud interfaces. This approach offers traceability and control in real-world environmental systems.

V. RESULTS

The proposed LSTM autoencoder effectively learned patterns in hourly water quality data. It achieved consistent reconstruction accuracy across most sequences. By using dynamic thresholding techniques, it first applied percentile-based cutoffs and later switched to an IQR-based strategy. This approach allowed the system to identify unusual sequences with high sensitivity while reducing false positives. The introduction of dropout regularization and bidirectional LSTM layers in Model 2 significantly improved its ability to generalize to new data.

During the simulation phase, five parallel TensorFlow Lite interpreters replicated the behavior of microcontroller units. They performed real-time inference on 50 sequential inputs. The system logged reconstruction errors and predictions to the Firebase Realtime Database. Anomalies were classified based on recent MSE distributions, and the system reliably responded to environmental changes. The accompanying web dashboard effectively visualized prediction trends, thresholds, and timestamps, offering a user-friendly interface for diagnosing anomalies.

VI. CONCLUSIONS

This study introduces a lightweight and deployable framework for unsupervised anomaly detection in water quality

monitoring with LSTM autoencoders. It combines strong time modeling, flexible statistical thresholding, and shared cloud integration. This system allows for scalable and clear diagnostics in edge-based applications. Converting to TFLite and simulating multi-device operation shows the model's practical use for embedded deployments. Future work may involve coupling predictive models with anomaly detectors, and enhancing the dashboard for geospatial mapping and historical trend analysis.

REFERENCES

- Design of Li River Water Quality Dynamic Monitoring System Based on Raspberry Pi and TinyML
- An energy efficient TinyML model for a water potability classification problem
- Safeguarding IoT consumer devices: Deep learning with TinyML driven real-time anomaly detection for predictive maintenance
- TinyML Meets IoT: A Comprehensive Survey
- An Intelligent IoT Sensing System for Rail Vehicle Running States Based on TinyML
- An Evolving TinyML Compression Algorithm for IoT Environments Based on Data Eccentricity
- A TinyML Deep Learning Approach for Indoor Tracking of Assets
- Tiny-Machine-Learning-Based Supply Canal Surface Condition Monitoring

ADDITIONAL RESOURCES

- **Colab Notebook 1:** Classification Model
- **Colab Notebook 2:** Regression Model
- **Colab Notebook 3:** Simulation
- **Google Cloud Vertex AI Console:** Autoencoder model with Simulation on GCP
- **Firebase Real-time Database:** Live Sensor Data View (Firebase)
- **Personal Logs and Notes:** Personal Logs and Observations

APPENDIX

Add the first page of Plagiarism Report here, after I provide the report to you (must have less than 15% similarity). Each team member should add their signature on the report page

project_report .pdf

ORIGINALITY REPORT

7 %

SIMILARITY INDEX

4 %

INTERNET SOURCES

2 %

PUBLICATIONS

5 %

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Aston University

Student Paper

1 %

2

Submitted to Imperial College of Science,
Technology and Medicine

Student Paper

1 %

3

www.dellemc.com

Internet Source

1 %

4

docs.oracle.com

Internet Source

1 %

5

Submitted to CSU, San Jose State University

Student Paper

1 %

6

Submitted to UNITEC Institute of Technology

Student Paper

1 %

7

docplayer.net

Internet Source

<1 %

8

www.i-scholar.in

Internet Source

<1 %
